# Reviewer Pack — Minimal p-Adic Order of Functions vs ACC⁰ Circuit Size for X…

Entry d1287f33f129 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 01:49:41 UTC

## Minimal p-Adic Order of Functions vs ACC⁰ Circuit Size for XOR

**Entry ID**: `d1287f33f129` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-24 01:49:41 UTC

### 1. Conjecture

**Field A** (mathematical branch): p-Adic Number Theory (Valuations) **Field B** (complexity object): Complexity Theory: ACC⁰ Circuit Complexity for XOR

**Statement**:

> ['For every explicit function f in P computing the XOR operation on n bits, there exists a prime p such that the p-adic order of f is at most Θ(n^(1/2)). Furthermore, if an ACC⁰ circuit with size S computes XOR, then the minimal prime p for which this property holds satisfies p ≥ 2^S.', 'For all explicit functions f in P computing XOR on n bits, there exists a constant c such that the p-adic order of f is at least cn^(1/4) for all primes p.']

**Rationale (proposer's reasoning)**:

> ['The minimal p-adic order of a function might capture essential information about its complexity, particularly when it comes to ACC⁰ circuits, which are known to be related to XOR.', 'p-Adic valuations have been used in other settings to analyze algorithms and complexity classes, suggesting that they could potentially offer new insights into ACC⁰ complexity.']

**Taxonomy category**: `ACC_SIPSER` (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix)**: `8b0b8b0ff7045327`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion**:

> The conjecture is supported if for all explicit functions f computing XOR on n bits, the minimal prime p such that the p-adic order of f is at most Θ(n^(1/2)) satisfies p ≥ 2^S and there exists a constant c such that the p-adic order is at least cn^(1/4) for all primes p. The conjecture is falsified if either condition fails.

### 3. Barrier filter (F1)

**Final**: `PASS`

| Barrier | Verdict | Confidence | LLM₁ | LLM₂ |
|---|---|---|---|---|
| RELATIVIZATION | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| NATURAL_PROOFS | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| ALGEBRIZATION | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |
| KARP_LIPTON | SAFE | 0.50 | UNCERTAIN | UNCERTAIN |

## 4. Novelty audit

**Verdict**: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - p-adic order function ACC⁰ circuit size XOR - 0(n^(1/2)) p-adic valuation ACC⁰ complexity - minimal prime p ≥ 2^S XOR p-adic order

## 5. Empirical test harness

**Multi-seed protocol**: 5 seeds (11, 23, 37, 53, 71)
**Sandbox**: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

### 5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def xor_function(n):
        return lambda x: sum(x[i] for i in range(n)) % 2

    def p_adic_order(f, p):
        if f(0) != 0:
            return 0
        n = 1
        while True:
            if f(p**n) == 0:
                n += 1
            else:
                break
        return n - 1

    def acc0_circuit_size(n):
        # Placeholder function for ACC⁰ circuit size calculation
        # This is a dummy implementation and should be replaced with an actual algorithm
        return 2**n

    results = []
    for n in [5, 10, 15, 20, 30, 40]:
        f = xor_function(n)
        p = random.choice([2, 3, 5, 7, 11, 13, 17, 19])
        order = p_adic_order(f, p)
        S = acc0_circuit_size(n)

        results.append({
```

```
            "n": n,
            "order": order,
            "S": S
        })

    mean_order = sum(result["order"] for result in results) / len(results)
    support_fraction = all(result["order"] <= math.sqrt(result["n"]) and result["p"] >= 2**result[

    return {
        "metric_name": "Minimal p-Adic Order vs ACC⁰ Circuit Size",
        "metric_value": mean_order,
        "instances_tested": len(results),
        "conjecture_holds": support_fraction,
        "counterexample": "" if support_fraction else "mapping_undefined"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_order = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_order} std=0.0 support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_f
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

## 6. Per-seed results

*(no seed-level data — test crashed before producing TRIAL: lines)*

## 7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7da46216.py", line 70, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7da46216.py", line 44, in run_trial
    order = p_adic_order(f, p)
            ^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7da46216.py", line 25, in p_adic_order
    if f(0) != 0:
       ^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7da46216.py", line 22, in <lambda>
```

```
    return lambda x: sum(x[i] for i in range(n)) % 2
                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7da46216.py", line 22, in <genexpr>
    return lambda x: sum(x[i] for i in range(n)) % 2
                         ~^^^
TypeError: 'int' object is not subscriptable
```

## 8. Critic adversarial review

**Critic verdict**: CHALLENGE

**Critic reasoning**:

> skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict**: INCONCLUSIVE

**Reasoning**:

> The test crashed before producing data, which means that it was unable to complete its execution and verify the conjecture. | next: Re-run the test with appropriate error handling to ensure it completes without crashing.

## 11. Audit log (LLM calls)

**Total LLM calls**: 10

| # | Phase | Provider | Model | Tokens in | out | Latency (ms) |
|---|---|---|---|---|---|---|
| 1 | propose | ollama_remote | glm4:latest | 0 | 0 | 12381 |
| 2 | propose | ollama_remote | glm4:latest | 0 | 0 | 10063 |
| 3 | preregistration | ollama_remote | glm4:latest | 0 | 0 | 6551 |
| 4 | novelty | ollama_remote | glm4:latest | 0 | 0 | 4635 |
| 5 | novelty | ollama_remote | glm4:latest | 0 | 0 | 5550 |
| 6 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 23123 |
| 7 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 10936 |
| 8 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 9640 |
| 9 | test_gen | ollama_remote | qwen2.5-coder:7b | 0 | 0 | 7932 |
| 10 | judge | ollama_remote | glm4:latest | 0 | 0 | 10750 |

**Totals**: 0 input tokens, 0 output tokens, ~$0.0000 cost, 101562 ms total latency. Provider mix: {'ollama_remote': 10}

*(full prompt+response transcripts available in `research/audit/d1287f33f129.jsonl`)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d1287f33f129.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file**: `research/pvsnp_sandbox/test_*.py` (per-cycle)
**Replay tarball**: `research/replay/d1287f33f129.tar.gz` (if generated)